WIKIPEDIA
25 years of the free encyclopedia

LLVM

LLVM is a set of compiler and toolchain technologies[5] that can be used to develop a frontend for any programming language and a backend for any instruction set architecture. LLVM is designed around a language-independent intermediate representation (IR) that serves as a portable, high-level assembly language that can be optimized with a variety of transformations over multiple passes.[6] The name *LLVM* originally stood for *Low Level Virtual Machine*. However, the project has since expanded, and the name is no longer an acronym but an orphan initialism.[7]

LLVM is written in C++ and is designed for compile-time, link-time, and runtime optimization. Originally implemented for C and C++, the language-agnostic design of LLVM has since spawned a wide variety of frontends: languages with compilers that use LLVM (or which do not directly use LLVM but can generate compiled programs as LLVM IR) include ActionScript, Ada, C# for .NET,[8][9][10] Common Lisp,[11] Crystal, CUDA, D,[12] Delphi,[13] Dylan, Forth,[14] Fortran,[15] FreeBASIC, Free Pascal, Halide, Haskell, Idris,[16] Jai (only for optimized release builds), Java bytecode, Julia, Kotlin, LabVIEW's G language,[17][18] Objective-C, OpenCL,[19] Odin,[20] PicoLisp, PostgreSQL's SQL and PL/pgSQL,[21] Ruby,[22] Rust,[23] Scala,[24][25] Standard ML,[26] Swift, Wolfram Language,[27] Xojo, and Zig.

LLVM

The LLVM logo, a stylized wyvern[1]

Original authors	Chris Lattner Vikram Adve
Developer	LLVM Developer Group
Release	2003
Stable release	22.1.8[2] / 16 June 2026
Written in	C++
Operating system	Cross-platform
Type	Compiler
License	Apache License 2.0 with LLVM Exceptions (v9.0.0 or later)[3] Legacy license:[4] UIUC (BSD-style)
Website	www.llvm.org (https://www.llvm.org)
Repository	github.com/llvm/llvm-project (https://github.com/llvm/llvm-project)

History

The LLVM project started in 2000 at the University of Illinois Urbana–Champaign, under the direction of Vikram Adve and Chris Lattner. LLVM was originally developed as a research infrastructure to investigate dynamic compilation techniques for static and dynamic programming languages. LLVM was released under the University of Illinois/NCSA Open Source License,[3] a permissive free software licence. In 2005, Apple Inc. hired Lattner and formed a team to work on the LLVM system for various uses within Apple's development systems.[28] LLVM has been an integral part of Apple's Xcode development tools for macOS and iOS since Xcode 4 in 2011.[29]

In 2006, Lattner started working on a new project named Clang. The combination of the Clang frontend and LLVM backend is named Clang/LLVM or simply Clang.

The name *LLVM* was originally an initialism for *Low Level Virtual Machine*. However, the LLVM project evolved into an umbrella project that has little relationship to what most current developers think of as a virtual machine. This made the initialism "confusing" and "inappropriate", and since 2011 LLVM is "officially no longer an acronym",[30] but a brand that applies to the LLVM umbrella project.[31] The project encompasses the LLVM intermediate representation (IR), the LLVM debugger, the LLVM implementation of the C++ Standard Library (with full support of C++11 and C++14[32]), etc. LLVM is administered by the LLVM Foundation. Compiler engineer Tanya Lattner became its president in 2014[33] and was still president and Executive Director as of November 2025.[34]

"For designing and implementing LLVM", the Association for Computing Machinery presented Vikram Adve, Chris Lattner, and Evan Cheng with the 2012 ACM Software System Award.[35]

The project was originally available under the UIUC license. After v9.0.0 released in 2019,[36] LLVM relicensed to the Apache License 2.0 with LLVM Exceptions.[3] As of November 2022 about 400 contributions had not been relicensed.[37][38]

Features

LLVM can provide the middle layers of a complete compiler system, taking intermediate representation (IR) code from a compiler and emitting an optimized IR. This new IR can then be converted and linked into machine-dependent assembly language code for a target platform. LLVM can accept the IR from the GNU Compiler Collection (GCC) toolchain, allowing it to be used with a wide array of extant compiler front-ends written for that project. LLVM can also be built with gcc after version 7.5.[39]

LLVM can also generate relocatable machine code at compile-time or link-time or even binary machine code at runtime.

LLVM supports a language-independent instruction set and type system.[6] Each instruction is in static single assignment form (SSA), meaning that each variable (called a typed register) is assigned once and then frozen. This helps simplify the analysis of dependencies among variables. LLVM allows code to be compiled statically, as it is under the traditional GCC system, or left for late-compiling from the IR to machine code via just-in-time compilation (JIT), similar to Java. The type system consists of basic types such as integer or floating-point numbers and five derived types: pointers, arrays, vectors, structures, and functions. A type construct in a concrete language can be represented by combining these basic types in LLVM. For example, a class in C++ can be represented by a mix of structures, functions and arrays of function pointers.

The LLVM JIT compiler can optimize unneeded static branches out of a program at runtime, and thus is useful for partial evaluation in cases where a program has many options, most of which can easily be determined unneeded in a specific environment. This feature is used in the OpenGL pipeline of Mac OS X Leopard (v10.5) to provide support for missing hardware features.[40]

Graphics code within the OpenGL stack can be left in intermediate representation and then compiled when run on the target machine. On systems with high-end graphics processing units (GPUs), the resulting code remains quite thin, passing the instructions on to the GPU with minimal changes. On systems with low-end GPUs, LLVM will compile optional procedures that run on the local central processing unit (CPU) that emulate instructions that the GPU cannot run internally. LLVM improved performance on low-end machines using Intel GMA chipsets. A similar system was developed under the Gallium3D LLVMpipe, and incorporated into the GNOME shell to allow it to run without a proper 3D hardware driver loaded.[41]

In 2011, programs compiled by GCC outperformed those from LLVM by 10%, on average.[42][43] In 2013, phoronix reported that LLVM had caught up with GCC, compiling binaries of approximately equal performance.[44]

Components

LLVM has become an umbrella project containing multiple components.

Frontends

LLVM was originally written to be a replacement for the extant code generator in the GCC stack,[45] and many of the GCC frontends were modified to work with it, resulting in the now-defunct LLVM-GCC suite. The modifications generally involved a GIMPLE-to-LLVM IR step so that LLVM optimizers and codegen could be used instead of GCC's GIMPLE system. Apple was a significant user of LLVM-GCC through Xcode 4.x (2013).[46]

This use of the GCC frontend was considered a temporary measure; it became mostly obsolete with Apple's development of Clang, a newer compiler frontend, with a more modern, modular codebase and greater compilation speed, supporting C, C++, and Objective-C. Support for LLVM-GCC was dropped in Xcode 5.[47] Primarily supported by Apple, Clang was aimed at replacing the C/Objective-C compiler in the GCC system with a system that is more easily integrated with integrated development environments (IDEs) and has wider support for multithreading. Support for OpenMP directives has been included in Clang since release 3.8.[48]

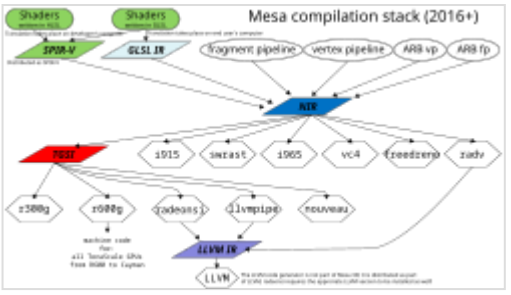
Widespread interest in LLVM has led to several efforts to develop new frontends for many other languages, including Ada, D, Delphi, Fortran, Haskell, Julia, Rust, Swift, and others.

The Utrecht Haskell compiler can generate code for LLVM. While the generator was in early stages of development, in many cases it was more efficient than the C code generator.[49] The Glasgow Haskell Compiler (GHC) backend uses LLVM and achieves a 30% speed-up of compiled code relative to native code compiling via GHC or C code generation followed by compiling, missing only one of the many optimizing techniques implemented by the GHC.[50]

Many other components are in various stages of development, including, but not limited to, a Java bytecode frontend, a Common Intermediate Language (CIL) frontend, the MacRuby implementation of Ruby 1.9, various frontends for Standard ML, and a new graph coloring register allocator.

Intermediate representation

The core of LLVM is the intermediate representation (IR), a low-level programming language similar to assembly. IR is a strongly typed reduced instruction set computer (RISC) instruction set which abstracts away most details of the target. For example, the calling convention is abstracted through `call` and `ret` instructions with explicit arguments. Also, instead of a fixed set of registers, IR uses an infinite set of temporaries of the form %0, %1, etc. LLVM supports three equivalent forms of IR: a human-readable assembly format,[51] an in-memory format suitable for frontends, and a dense bitcode format for serializing. A simple "Hello, world!" program in the human-readable IR format:



LLVM IR is used e.g., by radeonsi and by llvmpipe. Both are part of Mesa 3D.

```
@.str = internal constant [14 x i8] c"Hello, world\0A\00"

declare i32 @printf(ptr, ...)

define i32 @main(i32 %argc, ptr %argv) nounwind {
entry:
    %tmp1 = getelementptr [14 x i8], ptr @.str, i32 0, i32 0
    %tmp2 = call i32 (ptr, ...) @printf( ptr %tmp1 ) nounwind
    ret i32 0
}
```

The many different conventions used and features provided by different targets mean that LLVM cannot truly produce a target-independent IR and retarget it without breaking some established rules. Examples of target dependence beyond what is explicitly mentioned in the documentation can be found in a 2011 proposal for "wordcode", a fully target-independent variant of LLVM IR intended for online distribution.[52] A more practical example is PNaCl.[53]

The LLVM project also introduces another type of intermediate representation named MLIR[54] which helps build reusable and extensible compiler infrastructure by employing a plugin architecture named Dialect.[55] It enables the use of higher-level information on the program structure in the process of optimization including polyhedral compilation.

Backends

At version 16, LLVM supports many instruction sets, including IA-32, x86-64, ARM, Qualcomm Hexagon, LoongArch, M68K, MIPS, NVIDIA Parallel Thread Execution (PTX, also named *NVPTX* in LLVM documentation), PowerPC, AMD TeraScale,[56] most recent AMD GPUs (also named *AMDGPU* in LLVM documentation),[57] SPARC, z/Architecture (also named *SystemZ* in LLVM documentation), and XCore.

Some features are not available on some platforms. Most features are present for IA-32, x86-64, z/Architecture, ARM, and PowerPC.[58] RISC-V is supported as of version 7.

In the past, LLVM also supported other backends, fully or partially, including C backend, Cell SPU, mblaze (MicroBlaze),[59] AMD R600, DEC/Compaq Alpha (Alpha AXP)[60] and Nios2,[61] but that hardware is mostly obsolete, and LLVM developers decided the support and maintenance costs were no

longer justified.

LLVM also supports WebAssembly as a target, enabling compiled programs to execute in WebAssembly-enabled environments such as Google Chrome / Chromium, Firefox, Microsoft Edge, Apple Safari or WAVM. LLVM-compliant WebAssembly compilers typically support mostly unmodified source code written in C, C++, D, Rust, Nim, Kotlin and several other languages.

The LLVM machine code (MC) subproject is LLVM's framework for translating machine instructions between textual forms and machine code. Formerly, LLVM relied on the system assembler, or one provided by a toolchain, to translate assembly into machine code. LLVM MC's integrated assembler supports most LLVM targets, including IA-32, x86-64, ARM, and ARM64. For some targets, including the various MIPS instruction sets, integrated assembly support is usable but still in the beta stage.

Linker

The lld subproject is an attempt to develop a built-in, platform-independent linker for LLVM.[62] lld aims to remove dependence on a third-party linker. As of May 2017, lld supports ELF, PE/COFF, Mach-O, and WebAssembly[63] in descending order of completeness. lld is faster than both flavors of GNU ld.

Unlike the GNU linkers, lld has built-in support for link-time optimization (LTO). This allows for faster code generation as it bypasses the use of a linker plugin, but on the other hand prohibits interoperability with other flavors of LTO.[64]

C++ Standard Library

The LLVM project includes an implementation of the C++ Standard Library named libc++, dual-licensed under the MIT License and the UIUC license.[65]

Since v9.0.0, it was relicensed to the Apache License 2.0 with LLVM Exceptions.[3]

Polly

This implements a suite of cache-locality optimizations as well as auto-parallelism and vectorization using a polyhedral model.[66]

Debugger

C Standard Library

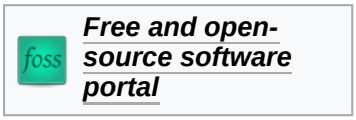
llvm-libc is an incomplete, upcoming, ABI independent C standard library designed by and for the LLVM project.[67]

Derivatives

Due to its permissive license, many vendors release their own tuned forks of LLVM. This is officially recognized by LLVM's documentation, which suggests against using version numbers in feature checks for this reason.[68] Some of the vendors include:

- AMD's AMD Optimizing C/C++ Compiler is based on LLVM, Clang, and Flang.
- Apple maintains an open-source fork for Xcode.[69]
- Arm provides a number of LLVM based toolchains, including Arm Compiler for Embedded targeting bare-metal development and Arm Compiler for Linux targeting the High Performance Computing market
- Flang, Fortran project in development as of 2022
- IBM is adopting LLVM in its C/C++ and Fortran compilers.[70]
- Intel has adopted LLVM for their next generation Intel C++ Compiler.[71]
- The Los Alamos National Laboratory has a parallel-computing fork of LLVM 8 named "Kitsune".[72]
- Nvidia uses LLVM in the implementation of its NVVM CUDA Compiler.[73] The NVVM compiler is distinct from the "NVPTX" backend mentioned in the Backends section, although both generate PTX code for Nvidia GPUs.
- Since 2013, Sony has been using LLVM's primary front-end Clang compiler in the software development kit (SDK) of its PlayStation 4 console.[74]

See also



- HHVM
- C--
- Amsterdam Compiler Kit (ACK)
- GNU lightning
- Pure
- ROCm
- Emscripten
- TenDRA Distribution Format
- Architecture Neutral Distribution Format (ANDF)
- Comparison of application virtualization software
- SPIR-V

References

1. "LLVM Logo" (https://llvm.org/Logo.html). *The LLVM Compiler Infrastructure Project*.
2. "LLVM 22.1.8" (https://github.com/llvm/llvm-project/releases/tag/llvmorg-22.1.8). June 16, 2026. Retrieved June 16, 2026.

3. "LICENSE.TXT" (https://releases.llvm.org/9.0.0/LICENSE.TXT). llvm.org. Retrieved September 24, 2019.
4. "LLVM Developer Policy — LLVM 20.0.0git documentation" (https://llvm.org/docs/DeveloperPolicy.html#legacy). *llvm.org*. Retrieved November 9, 2024.
5. "The LLVM Compiler Infrastructure Project" (https://llvm.org/). Retrieved March 11, 2016.
6. "LLVM Language Reference Manual" (https://llvm.org/docs/LangRef.html). Retrieved June 9, 2019.
7. "The LLVM Compiler Infrastructure Project" (https://llvm.org/). *llvm.org*. Retrieved January 13, 2025.
8. "Announcing LLILC - A new LLVM-based Compiler for .NET" (https://web.archive.org/web/20211212184833/https://dotnetfoundation.org/blog/2015/04/14/announcing-llilc-llvm-for-dotnet). *dotnetfoundation.org*. Archived from the original (https://dotnetfoundation.org/blog/2015/04/14/announcing-llilc-llvm-for-dotnet) on December 12, 2021. Retrieved September 12, 2020.
9. "Mono LLVM" (https://www.mono-project.com/Mono_LLVM). Retrieved March 10, 2013.
10. Lattner, Chris (2011). "LLVM" (https://www.aosabook.org/en/llvm.html). In Brown, Amy; Wilson, Greg (eds.). *The Architecture of Open Source Applications* (https://www.aosabook.org/).
11. "Clasp" (https://clasp-developers.github.io/). Clasp Developers. Retrieved December 2, 2024.
12. "LDC" (https://wiki.dlang.org/LDC). *D Wiki*. Retrieved December 2, 2024.
13. "LLVM-based Delphi Compilers" (https://docwiki.embarcadero.com/RADStudio/Sydney/en/LLVM-based_Delphi_Compilers). Embarcadero. Retrieved November 26, 2024.
14. "MovForth" (https://github.com/reschivon/movForth). *GitHub*. November 28, 2021.
15. "The Flang Compiler" (https://flang.llvm.org). LLVM Project. Retrieved December 2, 2024.
16. "Rapid" (https://rapid.sinyax.net/). *Rapid*. Retrieved November 22, 2024.
17. Wong, William (May 23, 2017). "What's the Difference Between LabVIEW 2017 and LabVIEW NXG?" (https://www.electronicdesign.com/test-measurement/what-s-difference-between-labview-2017-and-labview-nxg). *Electronic Design*.
18. "NI LabVIEW Compiler: Under the Hood" (https://www.ni.com/en/support/documentation/supplemental/10/ni-labview-compiler--under-the-hood.html).
19. Larabel, Michael (April 11, 2018). "Khronos Officially Announces Its LLVM/SPIR-V Translator" (https://www.phoronix.com/scan.php?page=news_item&px=SPIRV-LLVM-Translator). *Phoronix.com*.
20. "Frequently Asked Questions" (https://odin-lang.org/docs/faq/). *odin-lang.org*. Retrieved January 26, 2026.
21. "32.1. What is JIT compilation?" (https://www.postgresql.org/docs/11/jit-reason.html). *PostgreSQL Documentation*. November 12, 2020. Retrieved January 25, 2021.
22. "Features" (http://www.rubymotion.com/tour/features/). *RubyMotion*. Scratchwork Development LLC. Retrieved June 17, 2017. "RubyMotion transforms the Ruby source code of your project into ... machine code using a[n] ... ahead-of-time (AOT) compiler, based on LLVM."
23. "Code Generation - Guide to Rustc Development" (https://rustc-dev-guide.rust-lang.org/backend/codegen.html). *rust-lang.org*. Retrieved January 4, 2023.
24. Reedy, Geoff (September 24, 2012). "Compiling Scala to LLVM" (http://www.infoq.com/presentations/Scala-LLVM). St. Louis, Missouri, United States. Retrieved February 19, 2013.
25. "Scala Native" (https://scala-native.org/). Retrieved November 26, 2023.
26. "LLVMCodegen" (http://mlton.org/LLVMCodegen). *MLton*. Retrieved November 26, 2024.
27. Wolfram Language Documentation (https://reference.wolfram.com/language/ref/FunctionCompile.html)

28. Adam Treat (February 19, 2005), *mkspecs and patches for LLVM compile of Qt4* (https://web.archive.org/web/20111004073001/http://lists.trolltech.com/qt4-preview-feedback/2005-02/msg00691.html), archived from the original (http://lists.trolltech.com/qt4-preview-feedback/2005-02/msg00691.html) on October 4, 2011, retrieved January 27, 2012
29. "Developer Tools Overview" (https://web.archive.org/web/20110423095129/https://developer.apple.com/technologies/tools/). *Apple Developer*. Apple. Archived from the original (https://developer.apple.com/technologies/tools/) on April 23, 2011.
30. Lattner, Chris (December 21, 2011). "The name of LLVM" (https://lists.llvm.org/pipermail/llvm-dev/2011-December/046445.html). *llvm-dev* (Mailing list). Retrieved March 2, 2016.
" 'LLVM' is officially no longer an acronym. The acronym it once expanded too was confusing, and inappropriate almost from day 1. :) As LLVM has grown to encompass other subprojects, it became even less useful and meaningless."
31. Lattner, Chris (June 1, 2011). "LLVM" (https://www.aosabook.org/en/llvm.html). In Brown, Amy; Wilson, Greg (eds.). *The architecture of open source applications*. Lulu.com. ISBN 978-1257638017. "The name 'LLVM' was once an acronym, but is now just a brand for the umbrella project."
32. " "libc++" C++ Standard Library" (https://libcxx.llvm.org/).
33. Lattner, Chris (April 3, 2014). "The LLVM Foundation" (https://blog.llvm.org/2014/04/the-llvm-foundation.html). *LLVM Project Blog*.
34. "Board of Directors" (https://foundation.llvm.org/board-of-directors). *LLVM Foundation*. Retrieved September 18, 2025.
35. "ACM Software System Award" (https://awards.acm.org/software-system/award-winners?year=2012&award=149&region=&submit=Submit&isSpecialCategory=). ACM.
36. Wennborg, Hans (September 19, 2019). "[llvm-announce] LLVM 9.0.0 Release" (https://lists.llvm.org/pipermail/llvm-announce/2019-September/000085.html).
37. "Relicensing Long Tail" (https://web.archive.org/web/20240513154618/https://foundation.llvm.org/docs/relicensing_long_tail/). *foundation.llvm.org*. November 11, 2022. Archived from the original (https://foundation.llvm.org/docs/relicensing_long_tail/) on May 13, 2024. Retrieved April 1, 2022.
38. "LLVM relicensing - long tail" (https://docs.google.com/spreadsheets/d/18_0Hog_eSwES8lKwf7WJal3yBwwcYfvPu1yCfZnTcek/edit#gid=975215793). *Google Docs*. LLVM Project. Retrieved November 27, 2022.
39. "⚙ D156286 [docs] Bump minimum GCC version to 7.5" (https://reviews.llvm.org/D156286). *reviews.llvm.org*. Retrieved July 28, 2023.
40. Lattner, Chris (August 15, 2006). "A cool use of LLVM at Apple: the OpenGL stack" (https://lists.llvm.org/pipermail/llvm-dev/2006-August/006497.html). *llvm-dev* (Mailing list). Retrieved March 1, 2016.
41. Michael Larabel, "GNOME Shell Works Without GPU Driver Support" (https://www.phoronix.com/scan.php?page=news_item&px=MTAxMjI), *phoronix*, November 6, 2011
42. Makarov, V. "SPEC2000: Comparison of LLVM-2.9 and GCC4.6.1 on x86" (https://vmakarov.fedorapeople.org/spec/2011/llvmgcc32.html). Retrieved October 3, 2011.
43. Makarov, V. "SPEC2000: Comparison of LLVM-2.9 and GCC4.6.1 on x86_64" (https://vmakarov.fedorapeople.org/spec/2011/llvmgcc64.html). Retrieved October 3, 2011.
44. Larabel, Michael (December 27, 2012). "LLVM/Clang 3.2 Compiler Competing With GCC" (https://www.phoronix.com/scan.php?page=article&item=llvm_clang32_final). Retrieved March 31, 2013.
45. Lattner, Chris; Adve, Vikram (May 2003). *Architecture For a Next-Generation GCC* (https://llvm.org/pubs/2003-05-01-GCCSummit2003.html). First Annual GCC Developers' Summit. Retrieved September 6, 2009.
46. "LLVM Compiler Overview" (https://developer.apple.com/library/archive/documentation/CompilerTools/Conceptual/LLVMCompilerOverview/index.html). *developer.apple.com*.

47. "Xcode 5 Release Notes" (https://developer.apple.com/library/archive/documentation/Xcode/Conceptual/RN-Xcode-Archive/Chapters/xc5_release_notes.html). *Apple Inc*.
48. "Clang 3.8 Release Notes" (https://llvm.org/releases/3.8.0/tools/clang/docs/ReleaseNotes.html#openmp-support-in-clan). Retrieved August 24, 2016.
49. "Compiling Haskell To LLVM" (http://www.cs.uu.nl/wiki/bin/view/Stc/CompilingHaskellToLLVM). Retrieved February 22, 2009.
50. "LLVM Project Blog: The Glasgow Haskell Compiler and LLVM" (https://blog.llvm.org/2010/05/glasgow-haskell-compiler-and-llvm.html). May 17, 2010. Retrieved August 13, 2010.
51. "LLVM Language Reference Manual" (https://llvm.org/docs/LangRef.html). *LLVM.org*. January 10, 2023.
52. Kang, Jin-Gu. "Wordcode: more target independent LLVM bitcode" (https://llvm.org/devmtg/2011-09-16/EuroLLVM2011-MoreTargetIndependentLLVMBitcode.pdf) (PDF). Retrieved December 1, 2019.
53. "PNaCl: Portable Native Client Executables" (https://web.archive.org/web/20120502135033/http://nativeclient.googlecode.com/svn/data/site/pnacl.pdf) (PDF). Archived from the original (http://nativeclient.googlecode.com/svn/data/site/pnacl.pdf) (PDF) on 2 May 2012. Retrieved 25 April 2012.
54. "MLIR" (https://mlir.llvm.org/). *mlir.llvm.org*. Retrieved June 7, 2022.
55. "Dialects - MLIR" (https://mlir.llvm.org/docs/Dialects/). *mlir.llvm.org*. Retrieved June 7, 2022.
56. Stellard, Tom (March 26, 2012). "[LLVMdev] RFC: R600, a new backend for AMD GPUs" (https://lists.llvm.org/pipermail/llvm-dev/2012-March/048409.html). *llvm-dev* (Mailing list).
57. "User Guide for AMDGPU Backend — LLVM 15.0.0git documentation" (https://llvm.org/docs/AMDGPUUsage.html).
58. Target-specific Implementation Notes: Target Feature Matrix (https://llvm.org/docs/CodeGenerator.html#target-feature-matrix) // The LLVM Target-Independent Code Generator, LLVM site.
59. "Remove the mblaze backend from llvm" (https://github.com/llvm/llvm-project/commit/729866670b05108c399221ca3908400d94ef9783). *GitHub*. July 25, 2013. Retrieved January 26, 2020.
60. "Remove the Alpha backend" (https://github.com/llvm/llvm-project/commit/4c9fca99c9a6734bb33c34aeaf40b71c4002757e). *GitHub*. October 27, 2011. Retrieved January 26, 2020.
61. "[Nios2] Remove Nios2 backend" (https://github.com/llvm/llvm-project/commit/99fcbf67d04d488d819bffb8fda3bb9d5504b63b). *GitHub*. January 15, 2019. Retrieved January 26, 2020.
62. "lld - The LLVM Linker" (https://lld.llvm.org/). The LLVM Project. Retrieved May 10, 2017.
63. "WebAssembly lld port" (https://lld.llvm.org/WebAssembly.html).
64. "42446 – lld can't handle gcc LTO files" (https://bugs.llvm.org/show_bug.cgi?id=42446). *bugs.llvm.org*.
65. " "libc++" C++ Standard Library" (https://libcxx.llvm.org).
66. "Polly - Polyhedral optimizations for LLVM" (https://polly.llvm.org).
67. "llvm-libc: An ISO C-conformant Standard Library — libc 15.0.0git documentation" (https://libc.llvm.org/). *libc.llvm.org*. Retrieved July 18, 2022.
68. "Clang Language Extensions" (https://clang.llvm.org/docs/LanguageExtensions.html#feature-checking-macros). *Clang 12 documentation*. "Note that marketing version numbers should not be used to check for language features, as different vendors use different numbering schemes. Instead, use the Feature Checking Macros."
69. "apple/llvm-project" (https://github.com/apple/llvm-project). Apple. September 5, 2020.
70. "IBM C/C++ and Fortran compilers to adopt LLVM open source infrastructure" (https://developer.ibm.com/blogs/c-and-fortran-adopt-llvm-open-source/). July 29, 2022.
71. "Intel C/C++ compilers complete adoption of LLVM" (https://www.intel.com/content/www/us/en/develop/blogs/adoption-of-llvm-complete-icx.html). *Intel*. Retrieved August 17, 2021.

72. "lanl/kitsune" (https://github.com/lanl/kitsune). Los Alamos National Laboratory. February 27, 2020.
73. "NVVM IR Specification 1.5" (https://docs.nvidia.com/cuda/nvvm-ir-spec/index.html). "The current NVVM IR is based on LLVM 5.0"
74. *Developer Toolchain for ps4* (https://llvm.org/devmtg/2013-11/slides/Robinson-PS4Toolchain.pdf) (PDF), retrieved February 24, 2015

Further reading

- Chris Lattner - *The Architecture of Open Source Applications - Chapter 11 LLVM (https://www.aosabook.org/en/llvm.html)*, ISBN 978-1257638017, released 2012 under CC BY 3.0 (Open Access).[1]
- LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation (https://llvm.org/pubs/2004-01-30-CGO-LLVM.pdf), a published paper by Chris Lattner, Vikram Adve

External links

- Official website (https://llvm.org)

1. Lattner, Chris (March 15, 2012). "Chapter 11" (https://www.aosabook.org/en/llvm.html). *The Architecture of Open Source Applications*. Amy Brown, Greg Wilson. ISBN 978-1257638017.

Retrieved from "https://en.wikipedia.org/w/index.php?title=LLVM&oldid=1358674626"